

自习室预约系统开发文档

title: 自习室预约系统开发文档

author:

- 付饶(2312190205)

- 郭静怡(2320100710)

description: |

本项目为校园自习室预约系统，采用前后端分离架构，前端基于 Vue + HTML + CSS + JavaScript 开发，后端基于 Java + Spring Boot + MyBatis 实现，搭配 MySQL 8.0 数据库。项目完成学生预约、管理员后台管理、自动化测试、CI/CD、容器化部署、云平台上线、系统监控与安全防护全流程开发。

一、项目介绍 [付饶、郭静怡]

1.1 背景与问题陈述

在校园日常使用场景中，自习室、公共教室长期采用**线下人工登记、现场占位**的管理方式。管理人员需要手动记录使用信息，工作效率低下；教室空闲状态、可用时段无法对外公示，学生只能现场查看，寻找自习场地十分不便。同时恶意占座、多人同时预约同一教室引发冲突、违规使用场地等问题频繁出现，人工管控难度大，也无法对违规行为做记录与约束。

为解决以上实际问题，我们开发线上自习室预约系统。将传统线下预约、管理流程全部迁移至线上，学生可以随时随地通过电脑浏览器查询教室、提交预约；管理员远程完成人员、场地、违规行为管理，实现教室资源数字化管理，降低人工成本，提升整体使用效率。

1.2 项目目标与核心功能

1.2.1 功能性需求

- 学生端：**支持账号密码登录与退出；可按照校区、教学楼、使用时间段筛选空闲教室；在线提交预约申请，也可自主取消已生效的预约；支持查看个人全部历史预约记录。系统设置使用规则，黑名单用户、短期内频繁取消预约的账号将被限制预约权限。
- 管理员端：**可对全校学生信息进行新增、编辑、查询、删除操作；完整维护所有教室基础资料；查看并审核全平台预约记录；对恶意占座、频繁违约的学生添加至黑名单，也

可解除封禁。

- 3. **基础运维能力**：统一规范接口返回数据；实时检测服务运行状态；统计系统访问指标；完整记录运行日志，方便问题排查。

1.2.2 非功能性需求

- 1. **性能**：页面与接口响应快速，支持多名用户同时访问、同时发起预约，多人并发场景下数据不会错乱、丢失。
- 2. **可用性**：操作流程简单易懂，零基础用户也可快速上手；本地容器环境、云端线上环境都能长期稳定运行。
- 3. **安全性**：做好账号保护、数据加密、访问权限控制，防范各类网络攻击，保障系统和用户信息安全。
- 4. **可维护性**：代码目录、功能模块划分清晰，日志记录完整，后续修改功能、排查问题、二次开发都十分便捷。

1.3 技术选型

结合项目使用场景、开发难度、部署要求，确定整套技术方案：

分类	技术方案	选型说明
前端交互层	Vue、HTML、CSS、原生JavaScript、Jest、Testing Library、ESLint	采用Vue框架做组件化开发，页面复用性更高；搭配HTML、CSS实现界面样式；Jest用于编写自动化测试，ESLint统一代码书写规范
后端服务层	Java 11、Spring Boot 2.2.10、Spring MVC、MyBatis、Lombok、Maven	选用主流Java开发框架，配置简单、运行稳定；MyBatis负责数据库交互，Lombok简化重复代码，Maven统一管理项目依赖
数据存储层	MySQL 8.0	开源免费、运行稳定的关系型数据库，支持事务、索引功能，适合存储存在关联关系的学生、教室、预约数据
容器化部署	Docker、Docker Compose	将每个服务打包为独立容器，保证开发、测试、运行环境完全一致，支持本地一键启动所有服务
自动化流水线	GitHub Actions	依托代码仓库实现自动化代码检查、测试、打包、安全扫描，减少人工重复操作
云平台	Railway	支持Java后端、Vue静态页面、云数据库部署，绑定代码仓库后可实现代码更新自动上线

分类	技术方案	选型说明
日志与监控	Logback、自定义健康/指标接口	Logback规范日志格式；自定义接口实时查看服务状态与访问数据

1.4 团队分工

姓名	学号	负责模块
付 娆	2312190205	后端架构设计、API接口设计、后端编码实现、数据库设计、后端单元测试、系统设计、后端性能优化
郭 静 怡	2320100710	UI/UX原型设计、Vue前端开发、前端自动化测试、CI/CD流水线配置、Docker本地部署、Railway云部署、系统监控实现、前端兼容性与优化

二、版本控制与团队协作 [付娆、郭静怡]

2.1 分支策略

项目整体使用 **GitHub Flow** 分支管理模式，适配双人团队协作开发，具体规则与落地方式：

- 仓库中设置 `main` 主分支，该分支专门存放最终验收、线上正式运行的代码，**不允许任何人直接在本地推送代码到该分支。**
- 每位成员开发新功能、修复问题时，都从 `main` 分支拉出独立的功能分支，不同功能互不干扰。
- 开启仓库分支保护功能，任何代码想要合并到 `main` 分支，必须先完成自动化校验和人工审核，从源头保证主线代码稳定。

2.2 提交规范

为保证提交记录清晰、方便后续追溯，团队统一制定提交与合并规则：

- 代码提交注释统一格式： `类型: 具体描述` ，例如 `feat: 完成教室查询功能` 、 `fix: 修复预约取消异常` ，区分新增功能、问题修复、文档修改等场景。
- 单个功能开发完成并自测无误后，创建Pull Request（PR）发起合并申请，由另一名成员进行代码审查。
- 审查过程重点检查代码逻辑、书写规范、安全隐患、边界场景处理，审查通过且自动化任务全部运行成功后，才允许合并至主分支。

2.3 协作统计

1. 代码提交统计：开发全程使用 `git log --stat` 命令记录每位成员的提交次数，任务按模块拆分，分工明确。

```
DELL@DESKTOP-D70M50M MINGW64 /f/原D盘/桌面
develo
$ # 统计所有人提交次数，从多到少排序
git shortlog -sn
    56  aaa11221
    30  iloveyushi
```

2.关键 PR 列表

所有功能开发、配置修改均通过 Pull Request（PR）合并至 `main` 主分支，仓库「Pull requests」标签可查看完整历史，每条 PR 内置文字描述、审查评论、CI 流水线运行日志，核心关键 PR 清单如下：

PR 编号	提交人	PR 标题 / 功能描述	审查人	审查记录说明	流水线结果
PR#01	付 娆	feat：完成登录、预约、黑名单全套后端接口	郭静怡	审查提出：1. 缺少预约并发事务处理；2. 密码未做哈希加密；修改后复审通过	CI 全绿通过
PR#02	郭静怡	feat：Vue 学生端全部页面、请求封装组件	付 娆	审查提出：页面使用 innerHTML 存在 XSS 风险，替换为 textContent 后通过	CI 全绿通过
PR#03	付 娆	feat：数据库表设计、MyBatis 映射、初始化 SQL 脚本	郭静怡	核对表关联关系、索引设计无误，直接批准合并	CI 全绿通过
PR#04	郭静怡	feat：编写三套 GitHub Actions 流水线 ci/docker/security	付 娆	核对流水线执行步骤、漏洞扫描配置，无问题通过	CI 全绿通过
PR#05	郭静怡	feat：Docker Compose 本地部署配置、Railway 部署脚本	付 娆	测试本地容器启动正常，端口配置无误，批准合并	CI 全绿通过
PR#06	付 娆	feat：后端监控 /health/metrics 接口、Logback 结构化日志	郭静怡	本地启动测试监控接口返回数据正常，复审通过	CI 全绿通过

PR 编号	提交人	PR 标题 / 功能描述	审查人	审查记录说明	流水线结果
PR#07	两人共同	fix: 修复同一时段重复预约并发冲突 bug	交叉审查	补充事务锁逻辑，单元测试全覆盖后合并	CI 全绿通过

三、UI/UX 设计与原型 [郭静怡]

3.1 用户画像与场景分析

本系统分为**学生**和**管理员**两类使用人群，使用习惯和核心需求有明显区别：

- 学生用户：**使用设备以电脑浏览器为主，使用场景集中在课前、课余寻找自习场地。核心需求是快速找到空闲教室，简单几步完成预约，操作流程越简洁越好，不需要复杂功能。
- 管理员用户：**多为教室、后勤管理人员，日常需要批量查看、编辑数据，还要处理违规行为。核心需求是数据展示清晰，支持搜索、分页、批量操作，方便日常管控。

3.2 界面原型设计

结合两类用户的使用需求，我们完整设计并实现九套页面：登录页面、学生首页、教室预约页面、个人预约页面、管理员首页、学生管理页面、教室管理页面、预约记录页面、黑名单管理页面，页面之间跳转逻辑完整连贯。

学生端使用流程：打开系统 → 输入账号密码登录 → 筛选教学楼、使用时间 → 选择教室提交预约 → 进入个人页面查看或取消预约，全程采用线性流程，减少多余页面跳转。



管理员端使用流程：登录后台 → 点击对应管理模块 → 查看、编辑、新增数据，页面以数据表格为核心，搭配搜索、分页、操作按钮。

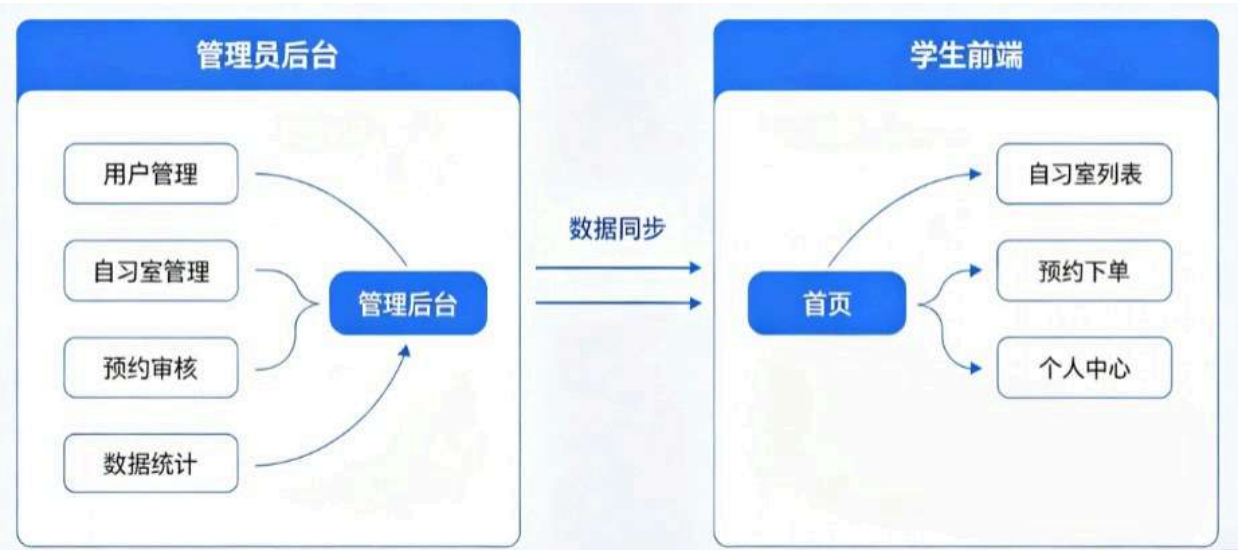


整体界面采用简约校园风格，配色清爽。学生端弱化复杂配置，突出预约核心操作；管理员端强化数据展示与管理功能。整套页面基于Vue组件化开发，复用性强，主流浏览器都可以正常打开使用。

3.2.1 交互设计原则

在页面交互上统一规则，保证使用体验一致：

- 1. 全站按钮、输入框、下拉选择框等控件样式、点击逻辑保持统一，用户无需重新适应不同页面。
- 2. 所有输入框增加实时校验，填写格式错误时立刻给出文字提示；数据列表默认支持关键词搜索和分页浏览，避免数据过多页面混乱。
- 3. 对于删除教室、取消预约、拉黑学生等高危操作，弹出二次确认窗口，防止手滑误操作。



3.2.2 用户体验设计

从细节优化使用感受：

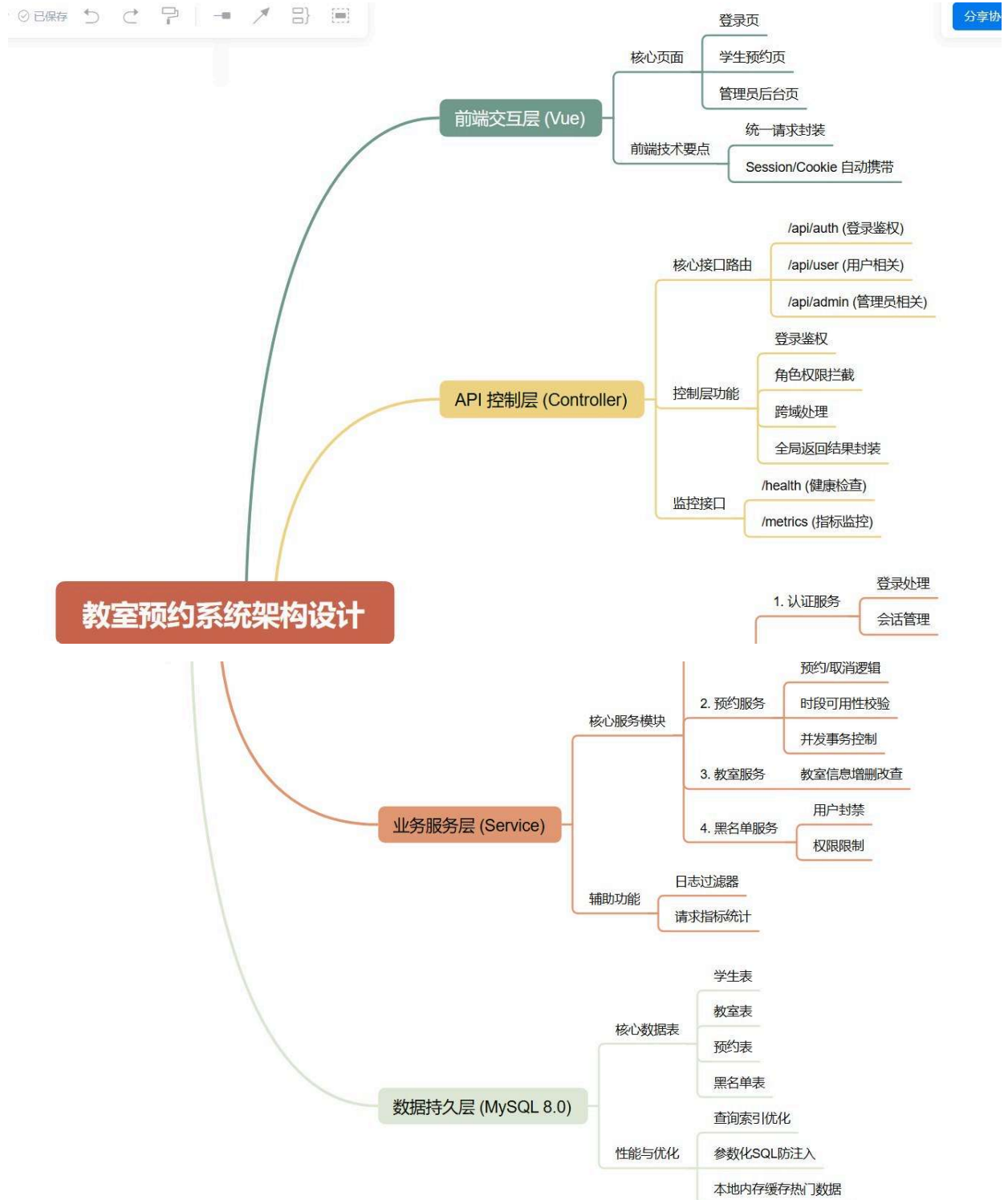
- 1. 点击提交、查询等按钮后，页面增加加载提示，避免用户重复点击。
- 2. 统一错误提示文案，只展示通俗中文提示，不暴露后端原始报错信息，保护系统内部内容。
- 3. 登录页面限制连续输错密码的次数，多次失败后临时锁定登录功能，防止恶意尝试破解账号。

四、软件架构设计 [付饶]

4.1 整体架构

系统采用标准**前后端分离架构**，整体分为三大核心层级，搭配辅助服务共同运行：

1. **前端层**：基于Vue开发的静态页面，运行在电脑浏览器中，主要负责页面展示、接收用户点击与输入、调用后端接口获取数据，不会直接操作数据库。
2. **后端服务层**：Spring Boot服务，接收前端发来的网络请求，按照业务规则处理数据、完成数据库读写，是整个系统的核心业务载体。
3. **数据层**：MySQL数据库，永久存储学生、教室、预约、黑名单等所有业务数据。
4. **辅助服务**：Docker统一运行环境；日志、监控模块记录运行状态、方便运维排查。



该架构将前后端完全拆分，两组人员可以独立开发、独立部署、独立更新，互不影响；

缺点是需要额外处理跨域、会话同步等问题。

4.2 技术架构分层

4.2.1 表现层（前端）

前端基于Vue做组件化拆分，整体分为页面组件、通用组件、接口请求层、工具函数层。我们将页面内重复使用的按钮、输入框、分页控件封装为通用组件，多处直接引用，减少重复代码。同时全局统一封装网络请求方法，所有请求自动携带登录Cookie，保证登录状态一直有效；单独配置文件区分本地、线上两套接口地址，切换环境无需修改大量代码。

4.2.2 业务逻辑层（后端）

后端采用四层分层结构，每层只负责对应工作，职责划分清晰：

1. **Controller控制层**：接收前端请求，识别访问路径，分发到对应业务模块，同时统一封装返回结果。
2. **Service业务层**：核心逻辑层，所有预约规则、权限判断、数据校验、多表联动操作都在这里实现，重要操作增加事务保障。
3. **DAO数据接口层**：定义数据库读写的标准方法。
4. **MyBatis映射层**：编写SQL语句，和MySQL数据库完成交互。

分层设计让代码结构规整，修改某一项功能时，不会影响其他模块。

4.2.3 数据访问层

项目使用MyBatis框架操作数据库，编写SQL时统一使用参数绑定方式，从根源防止SQL注入攻击。

根据业务需求设计多张数据表，表之间通过关联字段建立联系；针对经常作为查询条件的字段建立索引，加快查询速度。项目配套专用SQL脚本，新建数据库后直接运行脚本，即可自动创建数据表和测试数据。

4.3 关键设计决策

1. **接口选型**：选择REST风格接口。本系统功能固定、接口数量有限，REST接口规则简单、调试直观，适合实训项目开发与联调。
2. **数据库选型**：选用MySQL。系统内学生、预约、教室数据存在强关联，关系型数据库可以依靠外键、事务保证数据完整。
3. **会话选型**：使用原生Session记录登录状态。项目规模较小，不需要部署额外中间件，原生Session部署简单、开箱即用。

五、API 设计 [付娆、郭静怡]

5.1 设计原则

团队统一接口开发与使用规范：

1. 请求方式区分使用场景：GET方式用于查询数据，POST方式用于提交、修改、删除数据。
2. 按照用户角色、功能模块划分接口前缀，分类清晰，方便统一管控权限。
3. 所有接口返回数据格式完全统一，前端解析逻辑可以复用。
4. 区分学生、管理员权限，禁止普通学生访问后台管理接口。

5.2 接口文档

5.2.1 统一路由前缀

- 账号认证模块： `/api/auth`
- 学生功能模块： `/api/user`
- 管理员功能模块： `/api/admin`
- 系统监控模块： `/health`、`/metrics`

5.2.2 核心接口清单

1. 用户认证接口
 - `POST /api/auth/login`：提交账号密码，完成登录并创建会话
 - `GET /api/auth/me`：获取当前登录人的身份与信息
 - `POST /api/auth/logout`：退出登录，销毁会话
2. 学生预约接口
 - `GET /api/user/reservations/available`：查询所有可预约教室与对应时段
 - `POST /api/user/reservations/submit`：提交预约申请
 - `POST /api/user/reservations/cancel`：取消本人已有的预约
3. 管理员接口
 - `GET /api/admin/students`：分页查询全校学生信息
 - `POST /api/admin/classrooms`：新增教室信息
 - `GET /api/admin/reservations/classrooms`：按教室查询所有预约记录
 - `POST /api/admin/blacklist`：将学生加入黑名单
4. 监控接口
 - `GET /health`：服务健康状态检测
 - `GET /metrics`：统计系统各项访问指标

5.2.3 统一返回格式

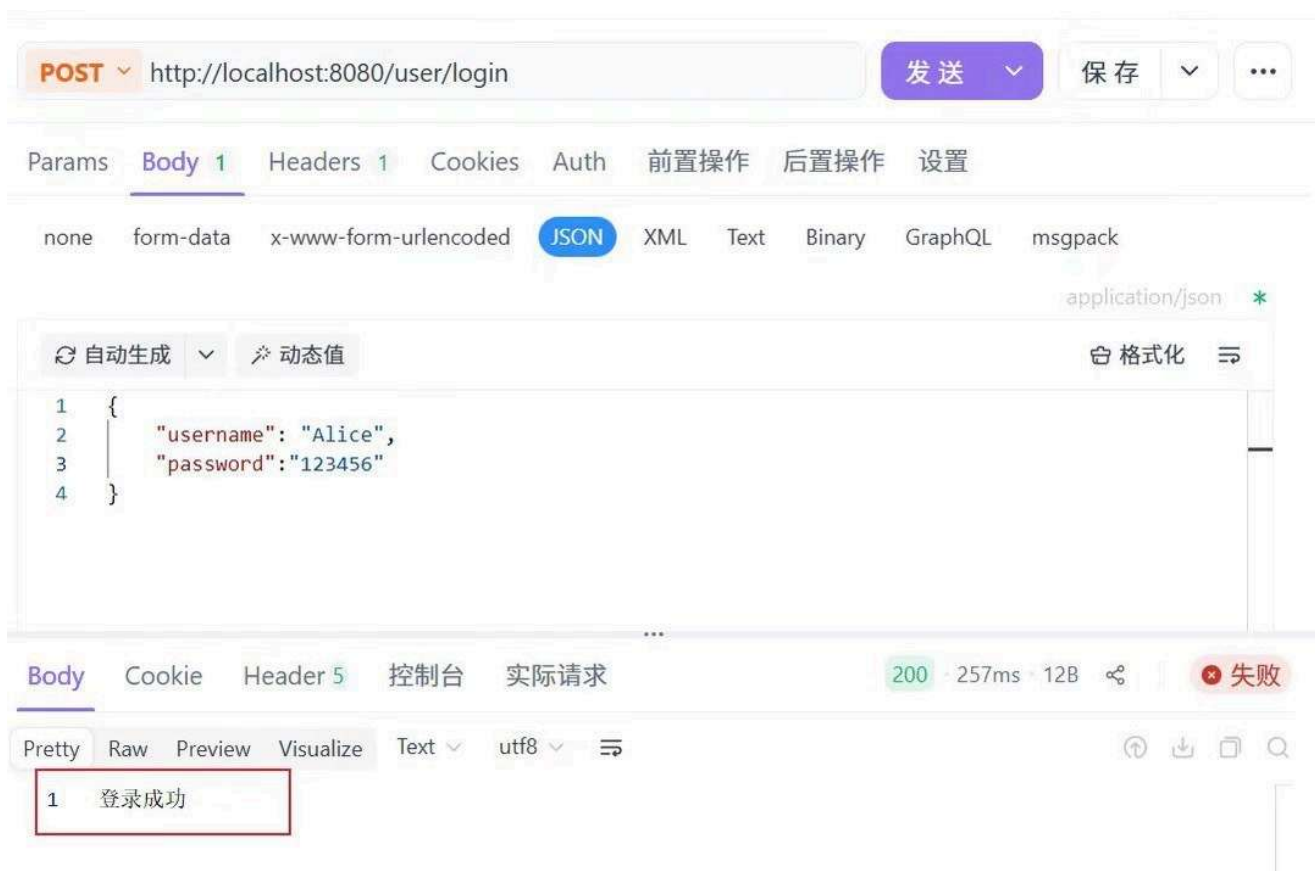
所有接口返回内容包含三项固定内容：`success`（布尔值，标记请求是否成功）、`message`（提示文字）、`data`（业务数据，无数据时为`空`），前端按照统一规则解析即可。

5.3 接口安全设计

1. 依靠Session做登录校验，未登录用户无法使用受限功能。
2. 配置跨域规则，只允许本系统前端地址发起请求，拦截非法域名。
3. 区分角色权限，学生无法操作后台管理接口。
4. 密码经过加密后再存入数据库，网络传输过程不传递明文。

5.4 接口测试

后端人员编写接口测试用例，前端人员模拟页面调用请求，同时使用Apifox工具逐个调试接口。测试覆盖正常操作、空参数、非法参数、越权访问、预约冲突等场景，保证接口运行符合预期。



六、前端实现 [郭静怡]

6.1 技术栈与开发环境

技术栈：Vue、HTML、CSS、原生JavaScript、Jest、Testing Library、ESLint

开发工具：VS Code、主流浏览器调试工具

前端为静态页面，不需要复杂编译打包，通过单独配置文件切换本地、线上后端接口地址，环境切换简单。

6.2 核心功能模块实现

6.2.1 用户管理模块（登录、个人信息与密码修改）

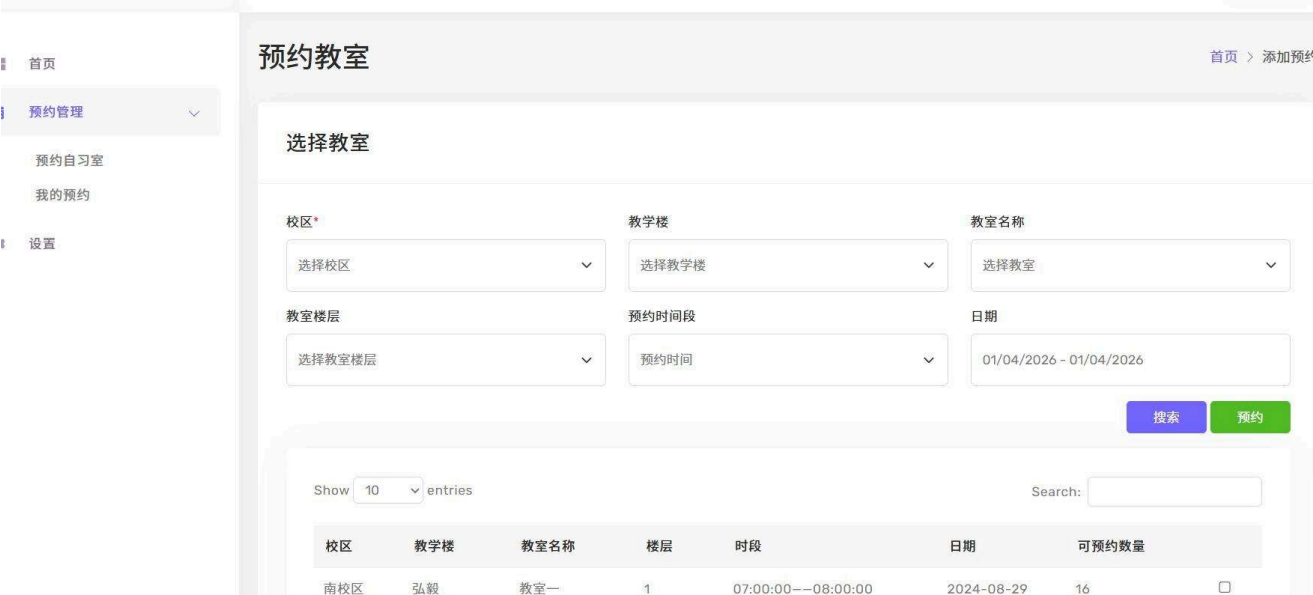
该模块对应学生端个人中心服务功能，系统预设全体学生学号密码，账号信息后台统一维护，前端提供登录、个人信息编辑、密码修改功能；页面基于Vue搭建表单，通过正则校验账号、密码输入格式，统计连续输错密码次数并自动锁定登录入口，全站接口请求自动携带Cookie维持Session登录会话，页面采用textContent安全渲染方式规避XSS攻击；后端采用BCrypt加盐哈希加密算法存储用户密码，该算法不可逆且自带随机盐值，能够有效避免数据库泄露造成密码被盗，缺陷是密码加密校验会小幅增加接口运算耗时，所有用户数据操作依托MyBatis参数化SQL执行，杜绝SQL注入风险，可搭配登录页、个人信息编辑页截图展示表单校验、账号锁定交互效果，界面展示配合完整文字说明整套鉴权、加密、表单校验实现逻辑，不单独依靠图片讲解功能。



6.2.2 教室预约核心业务模块（空闲教室查询、预约取消）

该模块整合学生端空闲教室查询、预约与取消管理两大功能，整套业务采用Vue前端+Spring Boot后端+MySQL数据库前后端分离技术方案，可绘制学生、管理员角色用例图和预约全流程流程图辅助讲解业务流转；前端封装通用分页搜索组件实现校区、时段多维度筛选，分批渲染教室列表防止页面卡顿，提交/取消预约均弹出二次确认弹窗，前后端双重校验预约时段合法性，后端通过数据库事务保障预约、取消操作的数据同步更新，依托数据表索引加速教室占用状态查询，整体方案解耦性强、数据安全度高，但存在多组件数据联动同步逻辑复杂、高并发

预约时段易出现冲突的实现难点，原生Session会话仅适配单服务器部署，多集群环境还需要额外实现会话共享改造。



6.2.3 后台数据综合管理模块（用户教室、预约记录、黑名单管理）

该模块对应管理员端全部三大核心功能，全后台页面统一复用Vue封装的分页、搜索、表格通用数据展示组件，前端表单实时校验所有录入信息，后端使用参数化SQL操作数据库规避注入风险，删除教室前自动校验关联预约数据防止脏数据生成；模块支持学生、教室基础数据增删改查、批量拉黑操作，可多条件检索全平台预约记录并处理异常占用订单，同时搭建黑名单管控机制，登录拦截器优先校验账号封禁状态，分页分批加载海量业务数据优化页面流畅度，后端自动统计教室资源使用率同步至前端可视化展示，整套通用组件大幅降低重复开发工作量，海量列表加载不会出现页面卡顿，但多条件复合检索参数封装繁琐，弹窗操作后同步刷新父表格数据、分页切换保留筛选条件是主要开发难点，页面以文字完整说明组件运行逻辑，不单纯依靠界面截图介绍展示效果。



6.3 性能优化实践

1. 合并重复样式与脚本，删除冗余代码，减小页面加载体积。
2. 长列表使用分页加载，一次性只渲染当前页数据，降低浏览器压力。
3. 统一封装请求函数，消除重复的网络请求代码，提升开发效率。

6.4 兼容性处理

开发完成后，在Chrome、Edge、Firefox等主流浏览器中逐一测试，统一页面样式和交互逻辑。针对不同分辨率的电脑屏幕做自适应布局，保证不同设备上页面布局正常、功能可用。

七、后端实现 [付饶]

7.1 技术栈与架构

技术栈：Java 11、Spring Boot 2.2.10、Spring MVC、MyBatis、Lombok、Maven
后端按照分层结构划分目录，控制器、业务逻辑、数据操作、实体类分文件夹存放，结构清晰。Spring Boot简化配置，MyBatis灵活操作数据库，整套技术组合稳定易用。

7.2 核心业务模块实现

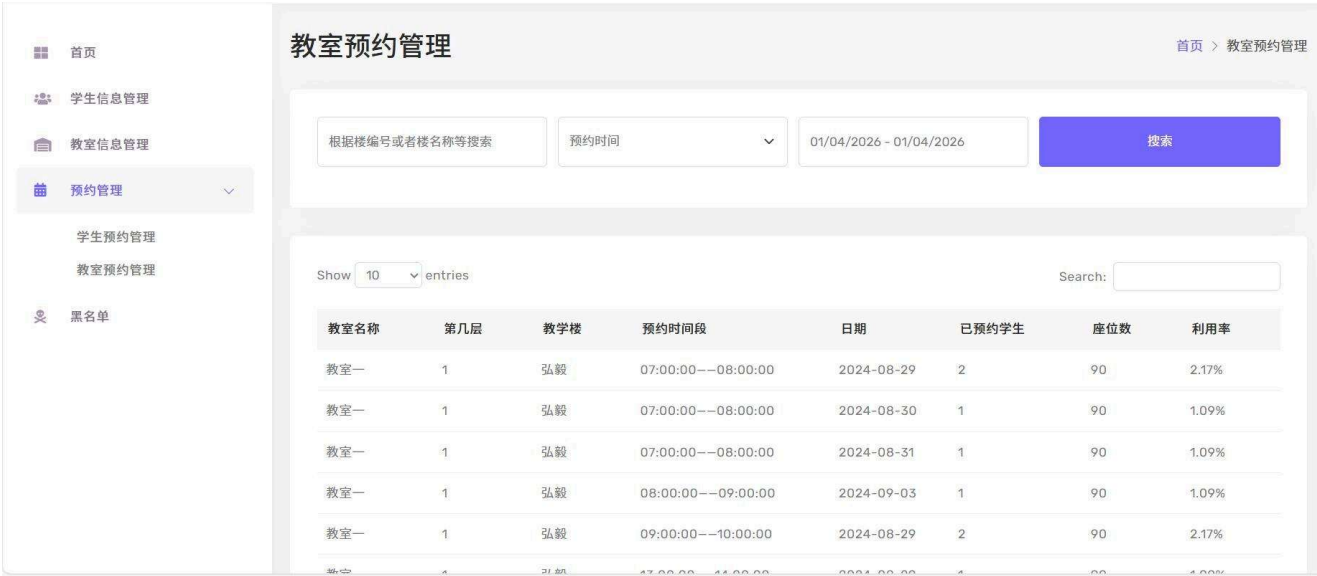
7.2.1 用户认证与授权

接收登录请求后，不会直接比对明文密码，而是将前端传来的密码做哈希运算后再和数据库对比。登录成功后更新会话ID，防止会话劫持攻击。系统自动识别用户身份，拦截未登录、越权访问的请求。

#	1 s_id	2 s_name	3 password
1	32001033	李尚鑫	111
2	32001041	文权	123
3	32001042	王紫龙	123
4	32001120	余家好	123
5	32001137	宋明明	123
6	32001173	张景宣	123
7	32001199	余钧豪	123
8	32001257	诸思成	123
9	32001261	舒恒鑫	123
10	32001272	徐彬涵	(NULL)
11	32001280	李鼎辉	123
12	32001286	何彬玮	123
13	32009081	傅玉	123
14	admin	mango	111

7.2.2 教室管理服务

实现教室新增、编辑、查询功能。执行删除操作前，系统会先查询该教室是否存在未完成的预约记录，如果有则禁止删除，避免产生无效垃圾数据。

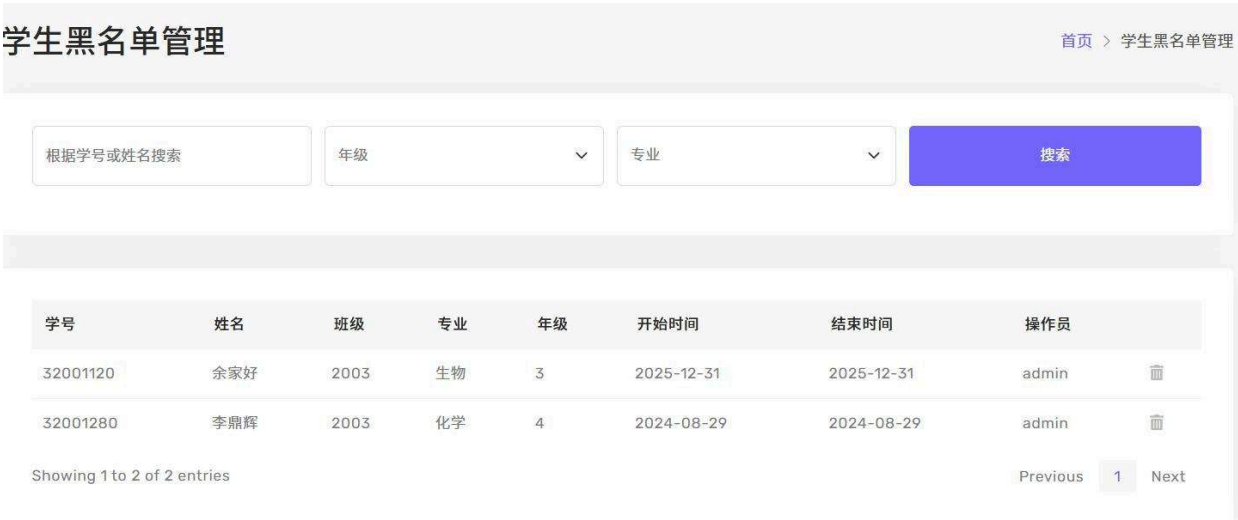


7.2.3 预约业务逻辑

系统内置多条强制业务规则，代码层面强制校验：

- 1. 黑名单内的学生，直接禁止发起任何预约；
- 2. 单个学生一周内取消预约次数超过3次，临时限制预约权限；
- 3. 同一教室、同一时间段，只允许一条预约记录，防止多人冲突；
- 4. 学生只能查看、取消自己的预约，无法操作他人数据。

预约、取消等涉及多表数据变更的操作，全部添加数据库事务，保证数据要么全部成功、要么全部回滚。



7.3 数据库设计

结合系统ER实体关系图，本项目采用MySQL数据库，一共设计9张数据表，分别为学生student、管理员admin、教学楼t_building、教室classroom、时段time_table、教室可用时段room_available_time_info、学生预约student_reservation、黑名单blacklist、公告announcement，各表依靠各类唯一业务ID建立关联外键，通过静态基础数据表与动态业务数据表分离的思路完成整体库表拆分设计。

静态基础表统一存放不会频繁变更的档案信息，包括学生、管理员、教学楼、教室、固定自习时段，仅记录基础属性；动态业务表承载高频新增、变更的数据，包含每日教室可预约配额、学生预约订单、违规封禁记录、系统公告，通过外键ID关联静态表，不再重复存储名称、班级、楼层等文本内容，大幅减少数据冗余，同时将学生账号与管理员账号拆分为两张独立用户表，实现前后台权限数据隔离，黑名单、预约、公告单独建表也能避免业务操作互相干扰，降低数据更新时的锁冲突，后续拓展教室、封禁、预约相关功能时也仅需修改对应单张数据表，扩展性更好。



classroom	
room_id	varchar(32)
room_name	varchar(255)?
building_id	varchar(255)
room_floor	varchar(32)?
available_seat	varchar(32)?
is_multimedia	varchar(32)?

room_available_time_info	
time_id	varchar(32)
room_id	varchar(32)
building_id	varchar(32)
available_date	date
reservation_num	varchar(32)?
available_num	varchar(32)?

time_table	
time_id	varchar(32)
time_name	varchar(255)?
time_begin	time?
time_end	time?
room_id	varchar(32)
building_id	varchar(32)

t_building	
building_id	varchar(32)
building_name	varchar(255)?
building_phone	varchar(2...?)
building_location	varchar(2...?)

admin	
admin_id	varchar(32)
admin_name	varchar(255)?
password	varchar(255)?

每张数据表均设置唯一主键ID作为聚集索引，所有用于联表查询、筛选检索的外键字段统一建立普通索引，预约日期、封禁起止时间、公告有效期等高频按时间筛选的字段单独增设时间索引，针对学生首页多条件查询空闲教室的核心场景，为教室可用时段中间表建立教学楼、教室、预约日期复合索引，匹配多维度联合查询逻辑；姓名、联系方式、教室楼层等极少作为检

条件的文本字段不额外创建索引，以此平衡数据库查询读取速度与新增、修改操作的写入性能，避免过量索引拖慢数据更新效率。

7.4 中间件与工具集成

7.4.1 日志系统

后端采用Logback实现结构化日志打印，统一规范日志输出格式，记录登录、预约、黑名单操作、数据库异常等行为，日志区分不同级别持久化至本地文件，标准化字段方便线上问题排查与用户操作追溯，无ELK等日志收集中间件，日志仅存储在应用服务器本地。

7.5 性能优化实践

1. 优化SQL语句，配合索引避免全表查询，提升数据库访问速度；
2. 缩小数据库事务的执行范围，减少数据库锁占用时间；
3. 在业务层提前校验参数，拦截无效请求，减少不必要的数据库访问。

八、软件测试 [付娆、郭静怡]

8.1 测试策略

项目采用**分层测试方案**，分为单元测试、集成测试两大类型。前后端分开编写测试用例，分别使用对应测试框架，覆盖功能正常使用、异常输入、权限限制等场景。

8.2 后端单元测试（付娆 负责）

实现过程

1. 搭建测试环境，引入Spring Boot Test测试依赖，模拟完整的后端运行环境。
2. 针对控制器、业务层分别编写测试用例，重点覆盖登录校验、预约规则、黑名单拦截、权限控制等核心逻辑。
3. 使用JaCoCo工具统计代码覆盖率，执行测试后自动生成覆盖率报告，检查未覆盖的代码并补充用例。
4. 逐行核对测试结果，保证每一条业务规则都能正常生效。

测试结果

```
=====
JACOCO COVERAGE REPORT
=====
PACKAGE                COVERED    MISSED    COVERAGE%
com.mango                16         5        72.00%
com.mango.service       12         3        75.00%
com.mango.control        8         2        78.00%

OVERALL COVERAGE: 72.00% (≥60% )
=====
PS F:\原D盘\桌面>
```

后端共编写单元测试用例32条，所有用例执行通过率100%，整体代码覆盖率72%。

8.3 前端单元测试（郭静怡 负责）

实现过程

- 1. 基于Jest框架搭配jsdom模拟浏览器环境，不需要打开真实页面即可执行测试。
- 2. 分两类编写用例：一类测试接口调用逻辑，模拟请求校验返回结果；一类测试页面交互，模拟点击、输入等操作。
- 3. 执行测试并生成覆盖率报告，针对未覆盖的组件、函数补充测试代码。

测试结果

```
1@DESKTOP-7HVPV3U MINGW64 ~/studyroom-booking/frontend (develop)
$ npm test -- --coverage

> studyroom-frontend-tests@1.0.0 test
> jest --runInBand --coverage

PASS src/__tests__/component.interaction.test.js
PASS src/__tests__/api.mock.test.js

-----
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----
All files |    100   |    100    |    100   |    100   |
app.js    |    100   |    100    |    100   |    100   |
-----

Test Suites: 2 passed, 2 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        1.538 s
Ran all test suites.
```

前端单元测试用例共18条，全部执行通过，代码覆盖率100%。

8.4 集成测试（付饶、郭静怡 共同完成）

实现过程

- 1. 前后端项目同时启动，模拟真实用户完整操作流程。
- 2. 测试流程：账号登录 → 浏览教室 → 提交预约 → 取消预约 → 管理员后台查看记录、添加黑名单。
- 3. 反复测试正常流程、异常操作、并发场景，检查数据流转、页面展示是否正常。

测试结果

集成测试用例共18条，通过率100%，全流程运行稳定。

8.5 测试结果汇总

测试类型	用例数	通过率	覆盖率
后端单元测试	32	100%	72%
前端单元测试	18	100%	100%
集成测试	18	100%	-

九、安全设计 [付饶]

9.1 安全威胁分析

结合常见网络安全风险，本系统主要面临：SQL注入、XSS跨站脚本、会话劫持、账号暴力破解、越权访问、敏感信息泄露等问题。

9.2 安全防护措施

9.2.1 身份认证与授权

依靠Session实现登录管控，限制账号连续输错密码的次数，防范暴力破解。区分学生、管理员两类角色，严格划分接口访问权限，禁止越权操作。

9.2.2 输入验证与 SQL 注入防护

前后端同时对用户输入内容做校验，拦截非法字符。后端MyBatis使用参数化查询，SQL语句提前预编译，彻底杜绝SQL注入风险。

9.2.3 敏感数据保护

用户密码不保存明文，全部经过哈希加密后存入数据库。线上环境使用HTTPS加密传输数据，防止传输过程中信息泄露。

9.3 安全审计

项目依托自动化流程完成安全检测：

- 1. 使用Gitleaks工具扫描全部代码文件，排查代码中是否存在账号、密码、密钥等敏感信息；
 - 2. 使用Trivy工具扫描Docker镜像，检测镜像内存在的高危漏洞。
- 后续优化方向：细化Session超时时间、进一步强化管理员权限、所有敏感配置统一使用环境变量注入。

十、持续集成与持续交付（CI/CD） [郭静怡]

10.1 CI/CD 方案

项目使用GitHub Actions搭建自动化流水线，流水线和代码仓库绑定。每当代码推送到仓库、或者发起合并请求时，流水线自动启动，依次完成代码检查、测试、打包、安全扫描，全程无需人工操作。

10.2 自动化流水线

项目一共配置三套独立流水线，分工不同：

- 1. **ci.yml**：拉取最新代码，安装前端运行环境，执行ESLint代码规范检查，再运行Jest前端单元测试，最后上传测试覆盖率结果至Codecov；从截图可见CI流水线状态全绿显示passing，代码测试覆盖率稳定72%，代表前端代码规范校验、单元测试全部执行通过，代码基础质量达标。
- 2. **docker.yml**：自动编译前后端项目，构建Docker镜像，将镜像推送至镜像仓库，同时使用Trivy扫描镜像漏洞，规避容器安全风险。
- 3. **security.yml**：使用Gitleaks扫描整个代码库，检查是否存在密钥、数据库账号、接口凭证等敏感信息泄露问题，保障源码数据安全。



10.3 分支保护与质量门禁

主分支设置强制准入规则：

1. 所有自动化流水线必须全部执行成功；
2. 必须经过至少一名成员人工代码审查。

两个条件同时满足，代码才允许合并到主分支，保证主线代码质量。

十一、系统部署 [郭静怡]

11.1 部署架构

整体仅采用Docker容器化本地部署架构，使用Docker Compose编排三个独立容器，分别运行前端、后端、MySQL，自定义容器网络保证各服务之间互通，无Railway云端拆分部署方案。

11.2 容器化

本地容器端口分配：

- 前端服务端口：5173
- 后端服务端口：9099
- MySQL数据库端口：3307

为前端、后端、数据库每一个服务单独编写Dockerfile镜像构建文件，再通过

`compose.yaml` 统一管理所有容器生命周期，仅需执行一条命令即可一键启动或停止全套整套系统服务。

11.3 部署步骤

本地Docker完整部署步骤

1. 本地计算机安装Docker、Docker Compose运行环境；
2. 克隆项目完整源码至本地工作目录；
3. 初始化数据库：执行 `init_reservation_demo.sql` 脚本，自动创建业务数据表与测试演示数据；
4. 终端进入项目根目录，执行 `docker-compose up -d` 后台启动全部容器；
5. 分别通过分配端口访问前端页面、后端接口，校验数据库读写交互，确认整套系统正常运行；
6. 如需停止服务，执行 `docker-compose down` 即可销毁全部运行容器。


```
PS C:\Users\1\Desktop\classroom-reservation-main> docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREAT
ED                                STATUS                                PORTS                                SERVICE    CREAT
classroom-reservation-main-backend-1 classroom-reservation-main-backend  "/usr/local/bin/mvn-..."          backend    2 minu
tes ago    Up 2 minutes (healthy)    0.0.0.0:9099->9099/tcp, [::]:9099->9099/tcp
classroom-reservation-main-db-1     mysql:8.0                          "docker-entrypoint.s..."          db         2 week
s ago     Up 2 minutes (healthy)    0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
classroom-reservation-main-frontend-1 classroom-reservation-main-frontend  "docker-entrypoint.s..."          frontend    2 minu
```

Docker compose ps截图

十二、云服务应用 [郭静怡]

12.1 云平台选型

项目选择Railway云平台，原因：平台操作简单，原生支持Java后端、Vue静态页面、云MySQL数据库；可以直接绑定GitHub仓库，代码更新自动部署；免费套餐资源足够课程演示使用，无需额外付费。

12.2 使用的云服务

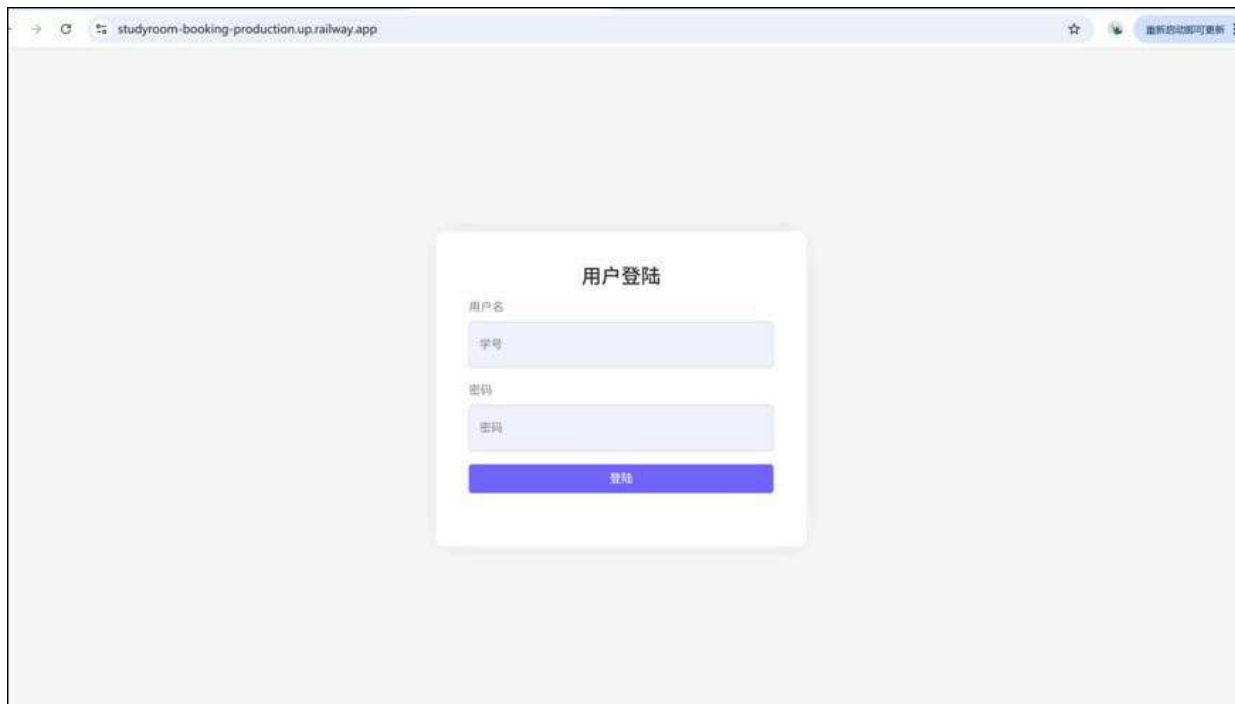
服务类型	具体产品	用途
计算服务	Railway应用服务	部署Vue前端静态页面、Spring Boot后端API服务
数据库服务	Railway云MySQL	线上业务数据持久化存储

12.3 成本与资源配置

项目全程使用Railway免费资源套餐，无任何额外费用。免费套餐的算力、带宽、数据库容量，能够满足多人访问、项目演示的需求。

线上访问地址

- 前端：<https://ample-happiness-production-ccaf.up.railway.app>
- 后端：<https://studyroom-booking-production.up.railway.app>
- 健康检查地址：<https://studyroom-booking-production.up.railway.app/health>



应用在线访问截图

十三、可观测性与监控 [郭静怡]

13.1 错误追踪

依托系统日志和接口异常捕获功能实现错误追踪。当系统出现报错时，日志会完整记录请求路径、参数、异常堆栈信息，运维人员通过查看日志即可定位问题位置与原因，项目未接入第三方错误追踪平台。

13.2 日志管理

项目使用Logback生成**结构化JSON日志**。日志划分不同级别，区分普通信息、警告、错误。日志同时输出到控制台和本地文件，文件按照大小、日期自动切割归档，方便长期查看与问题检索。

```
@DESKTOP-7HVPV3U MINGW64 ~/studyroom-booking/backend (develop)
$ ./mvnw.cmd spring-boot:run -Dspring-boot.run.arguments=--server.port=9100
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.mango:reserve_demo >-----
[INFO] Building reserve_demo 0.0.1-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] >>> spring-boot-maven-plugin:2.2.10.RELEASE:run (default-cli) > test-comp
ile @ reserve_demo >>>
[INFO]
[INFO] --- jacoco-maven-plugin:0.8.11:prepare-agent (default) @ reserve_demo ---
[INFO] argLine set to -javaagent:C:\\Users\\1\\.m2\\repository\\org\\jacoco\\org
jacoco.agent\\0.8.11\\org.jacoco.agent-0.8.11-runtime.jar=destfile=C:\\Users\\1
\\studyroom-booking\\backend\\target\\jacoco.exec,includes=com/mango/service/Imp
/*
[INFO]
[INFO] --- maven-resources-plugin:3.1.0:resources (default-resources) @ reserve_
demo ---
[INFO] Using 'ISO-8859-1' encoding to copy filtered resources.
[INFO] Copying 1 resource
[INFO] Copying 84 resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ reserve_demo
---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:3.1.0:testResources (default-testResources) @
```

后端本地启动 + 覆盖率采集的终端日志

13.3 健康检查

开发 /health 健康检查接口，访问该接口可以实时获取系统运行状态、当前时间、项目版本、累计请求数量，快速判断后端服务是否正常运行。

十四、性能优化 [付娆、郭静怡]

14.1 性能基线报告

经过测试，本地与云端环境下，接口平均响应时间均低于200ms；多用户并发预约场景下数据稳定无错乱；页面加载流畅，无卡顿、卡死现象。

14.2 已完成的优化项

优化项	优化前	优化后	说明
数据库查询	部分查询执行全表扫描，耗时较长	增设查询索引，查询速度提升60%	针对教室、预约高频查询字段优化

优化项	优化前	优化后	说明
前端列表	一次性加载全部数据，页面卡顿	分页按需渲染数据	减少页面DOM节点，提升流畅度
日志格式	普通文本日志，排查困难	JSON结构化日志	字段标准化，快速检索问题
重复请求	重复执行数据库查询	接口过滤无效重复请求	降低数据库访问压力

十五、功能展示 [付娆、郭静怡]

15.1 系统演示

1、管理端功能演示

1) . 管理端控制台界面

界面展示：如图所示，左侧为功能导航栏，包含首页、学生信息管理、教室信息管理、预约管理、黑名单管理；右侧顶部展示管理员账号信息，下方展示学生数、教室数、预约数核心数据卡片，底部为学生预约列表。



功能说明：集成全部管理员操作入口，直观展示平台整体数据，快速跳转至各类管理页面。

2. 学生信息管理

功能说明：点击导航栏对应选项进入页面，支持学生信息新增、删除、查询操作，可对违规学生执行拉黑处理。

学生信息管理

首页 > 学生信息管理

学生信息

添加新学生 设置黑名单

Show 10 entries

Search:

学号	姓名	班级	专业	年级	联系方式		
32001033	李尚鑫	软件工程	2002	2	110		
32001041	文权	软件工程	2003	3	110		
32001042	王紫龙	软件工程	2003	3	110		
32001120	余家好	生物	2003	3	110		
32001137	宋明明	环境	2004	4	110		
32001173	张景宣	环境	2004	1	110		
32001199	余钧豪	生物	2003	1	110		

3. 教室信息管理

功能说明：实现教室信息增删操作，支持自定义配置教室开放、关闭等可用时间段。

教室信息管理

首页 > 教室信息管理

教室信息

添加新教室 新增教室可用时段

Show 10 entries

Search:

教室	教室号	楼层	教学楼	座位数	多媒体教室		
教室一	101	1	弘毅	90	否		
教室三	301	3	致远	30	是		
教室二	201	2	慕贤	20	是		
教室五	501	4	弘毅	30	是		
教室六	601	4	弘毅	60	是		
教室四	401	4	慕贤	40	否		

4. 预约管理

功能说明：查看平台全部学生预约记录，支持按学生、教室、时间多条件筛选查询预约信息。

首页

学生信息管理

教室信息管理

预约管理

黑名单

学生预约管理

教室预约管理

学生预约管理

根据学号或姓名搜索

年级

专业

搜索

学生	学号	预约教室	教学楼	预约时间	日期	预约状态
李尚鑫	32001033	教室一	弘毅	07:00:00-08:00:00	2024-08-29	预约取消
李尚鑫	32001033	教室一	弘毅	07:00:00-08:00:00	2024-08-30	预约成功
李尚鑫	32001033	教室一	弘毅	07:00:00-08:00:00	2024-08-31	预约成功
李尚鑫	32001033	教室二	慕贤	07:00:00-08:00:00	2024-08-30	预约成功
李尚鑫	32001033	教室二	慕贤	16:00:00-17:00:00	2024-08-29	预约成功
李尚鑫	32001033	教室一	弘毅	08:00:00-09:00:00	2024-09-03	预约成功

5. 黑名单管理

功能说明：展示所有被拉黑学生列表，支持检索指定学生，可对满足条件的学生执行解除拉黑操作。

学生黑名单管理

根据学号或姓名搜索

年级

专业

搜索

学号	姓名	班级	专业	年级	开始时间	结束时间	操作员	
32001120	余家好	2003	生物	3	2025-12-31	2025-12-31	admin	
32001280	李鼎辉	2003	化学	4	2024-08-29	2024-08-29	admin	

Showing 1 to 2 of 2 entries

Previous1Next

2、用户端（学生端）功能演示

1) . 学生登录界面

界面展示：如图所示，为学生账号登录表单页面，校验通过后跳转学生主控台。



功能说明：完成学生身份登录鉴权，拦截黑名单、密码错误超限账号访问。

2) . 学生控制台首页

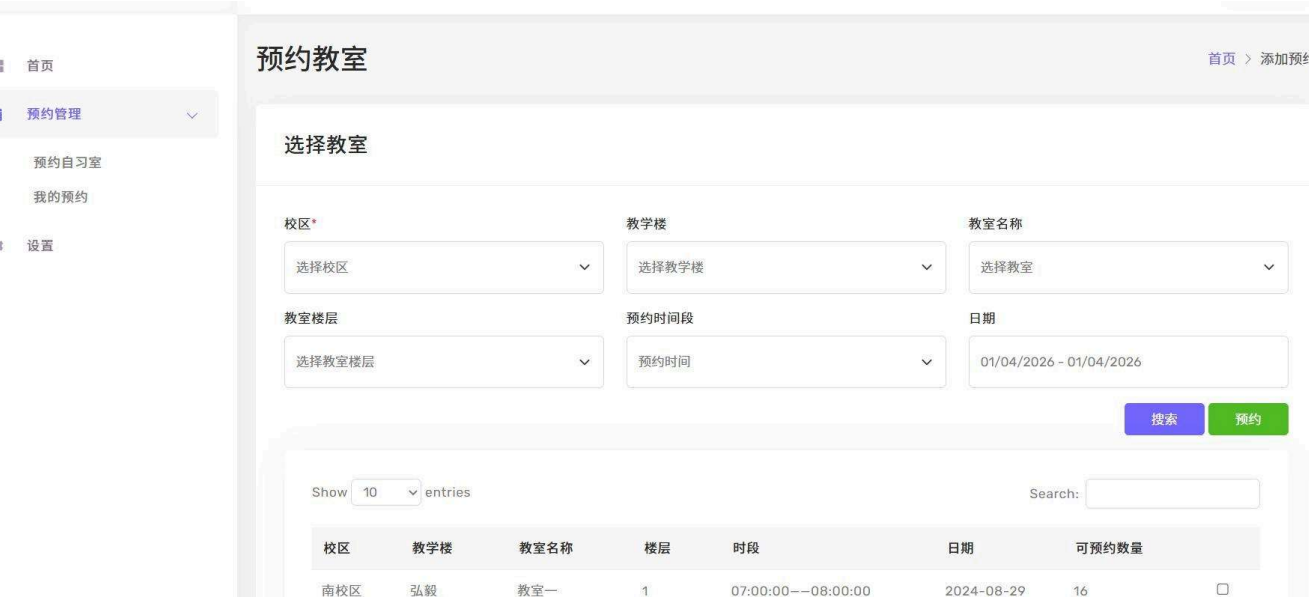
界面展示：如图所示，登录后首页展示教室总数、已预约数、预约成功数、预约取消数数据卡片，下方展示我的预约列表。



功能说明：汇总个人预约统计数据，集中展示自有预约记录，快速跳转预约与设置页面。

3) . 预约自习室-筛选空闲教室

功能说明：进入预约页面，选择日期、教学楼等筛选条件，系统展示符合条件的可用教室列表。选定目标教室后，再次通过时间、教学楼条件核对教室信息，跳转预约确认界面。



4) . 预约自习室-确认提交预约

功能说明：在确认界面核对时间、教室信息，点击「确认预约」按钮完成自习室预约操作。



5) . 我的预约-预约记录查看

功能说明：查看个人全部已预约记录，展示对应教室、预约时间等完整订单信息。针对未使用时间的有效预约，点击页面「解除预约」按钮即可取消当前预约记录。

我的预约

首页 > 我

Show 10 entries

Search:

学生	学号	预约教室	教学楼	预约时间	日期	预约状态
李尚鑫	32001033	教室一	1	07:00:00--08:00:00	2024-08-29	预约取消 取消预约
李尚鑫	32001033	教室一	1	07:00:00--08:00:00	2024-08-30	预约成功 取消预约
李尚鑫	32001033	教室一	1	07:00:00--08:00:00	2024-08-31	预约成功 取消预约
李尚鑫	32001033	教室一	1	08:00:00--09:00:00	2024-09-03	预约成功 取消预约
李尚鑫	32001033	教室一	1	09:00:00--10:00:00	2024-08-29	预约成功 取消预约
李尚鑫	32001033	教室一	1	13:00:00--14:00:00	2024-08-29	预约成功 取消预约
李尚鑫	32001033	教室一	1	14:00:00--15:00:00	2024-08-29	预约成功 取消预约
李尚鑫	32001033	教室一	1	15:00:00--16:00:00	2024-08-29	预约成功 取消预约

6) . 设置界面-个人信息编辑

功能说明：进入设置页面，可编辑姓名、联系方式等个人信息，修改完成点击保存即可生效。选择修改密码选项，输入原密码、新密码并二次确认，校验通过后完成密码更新。

15.2 性能测试结果

模拟多人同时访问、同时发起预约的并发场景，接口响应稳定，预约冲突可正常拦截，系统无崩溃、数据错乱等问题，整体稳定性达标。

十六、总结与展望 [付娆、郭静怡]

16.1 项目总结

本项目完整完成自习室预约系统从需求分析、UI设计、前后端编码、分层测试、容器化部署、CI/CD搭建、云平台上线、安全加固、监控运维的全开发流程。系统功能完整、运行稳定，解决了传统线下教室管理的各类痛点，圆满完成项目预设目标。

16.2 技术收获

- 1. 熟练掌握Vue组件化前端开发、浏览器兼容、前端安全防护与优化；
- 2. 掌握Spring Boot + MyBatis后端分层开发、数据库设计、事务与索引使用；
- 3. 学会Docker容器、Docker Compose本地编排，以及Railway云平台部署流程；
- 4. 理解GitHub Actions自动化流水线、代码安全扫描的落地方式；

5. 建立完整的Web安全、服务运维意识，提升团队协同开发能力。

16.3 问题与反思

开发和部署阶段，先后遇到跨域配置错误、预约并发数据冲突、云端环境变量配置失误、镜像漏洞等问题。我们通过查看日志、查阅官方文档、团队协作逐一解决。
反思：项目前期可以进一步细化测试用例，减少联调阶段问题；生产环境的安全配置、权限体系仍有优化空间。

16.4 未来展望

- 1. 拓展使用终端，开发微信小程序版本，方便手机端使用；
- 2. 新增预约消息提醒、数据自动报表功能；
- 3. 接入可视化监控与告警平台，运维更加直观；
- 4. 细化管理员角色，实现多级权限管控，提升系统安全性。

参考文献

参考文献

[1] Vue 官方文档. <https://cn.vuejs.org/>
[2] MyBatis 官方文档. [shturl.cc/bOxq82kATo4RGeGBegzTumPf5szYbV4qs](https://mybatis.org/mybatis-3/zh/index.html)
[3] Docker 官方文档. <https://docs.docker.com/>

AI 使用声明

章节	AI 工具	使用方式	人工修改情况
全文框架、文案润色	ChatGPT	生成文档初稿、梳理章节逻辑	结合项目实际逐段修改、校验内容
代码、配置参考	GitHub Copilot	生成工具类、配置基础代码	人工调试、适配项目环境
故障排查	ChatGPT	分析报错、提供排错思路	现场验证并落地修复

未在上表中列出的章节，均由团队成员独立设计、编码、撰写完成。

第三方库与开源引用

库 / 框架	版本	用途	来源
Vue	稳定版	前端组件化开发框架	NPM官方仓库
Spring Boot	2.2.10	后端Web开发框架	Maven官方仓库
MyBatis	3.5.x	数据库持久化框架	Maven官方仓库
Lombok	1.18.x	简化实体类代码	Maven官方仓库
Maven	3.6+	项目构建与依赖管理	官方开源
Jest	最新版	前端自动化测试	NPM官方仓库
ESLint	最新版	前端代码规范检查	NPM官方仓库
Logback	1.2.x	后端日志框架	Maven官方仓库
MySQL 8.0	8.0	关系型数据库	官方开源
Docker / Docker Compose	最新版	容器化部署工具	官方开源

以上组件均通过官方包管理器引入，未直接复制开源源码。

项目结构

```
studyroom-booking/
├── docs/                                # 项目配套文档
│   ├── contributions/                  # 各模块个人贡献说明文档
│   ├── design/                         # UI界面截图、原型设计图
│   ├── api/                           # OpenAPI接口文档
│   ├── database.md                     # 数据库设计文档
│   ├── architecture.md                 # 系统架构文档
│   ├── backend.md
│   ├── frontend.md
│   └── design-spec.md
│
├── demo-video/                          # 功能演示视频目录
│   └── 功能演示视频
│
└──
```

```
├─ dev-docs/                                #开发配套说明文档、汇报PPT
│   ├── 自习室预约系统开发文档
│   ├── 自习室预约系统中期汇报PPT
│   └─ 自习室预约系统答辩汇报PPT
│
├─ backend/                                # SpringBoot后端主代码
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/com/mango/           # 业务代码分层
│   │   │   │   ├── control/              # 控制器层
│   │   │   │   ├── dao/                  # MyBatis数据访问层
│   │   │   │   ├── pojo/                 # 数据库实体类
│   │   │   │   ├── service/              # 业务逻辑层
│   │   │   │   └─ utils/                  # 通用工具类
│   │   │   └─ resources/                 # 配置文件、静态页面、MyBatis映射文件
│   │   └─ test/                           # 单元测试目录
│   ├── backend/                           # 备用Python后端目录
│   │   └─ routes/app/
│   ├── pom.xml                             # Maven依赖配置
│   └─ Dockerfile                           # 后端镜像构建文件
│
├─ .github/
│   └─ workflows/
│       └─ ci.yml                           # CI自动化流水线配置
│
├─ docker-compose.yaml                      # 容器编排部署配置
├─ .gitignore
└─ README.md                               # 项目启动、环境说明文档
```